

Predicting Customer Churn in a Telecommunications Company

A Machine Learning Pipeline with Deployment

Mohammed S. Rahman

May 2026

Abstract

This report documents the design, implementation, and evaluation of a supervised machine learning pipeline for predicting customer churn in a telecommunications company. Using a publicly available dataset of 7,043 customer records, the pipeline progresses through data cleaning, feature engineering, preprocessing, model training, and deployment as an interactive web application. Three model classes are trained and compared—Logistic Regression, Random Forest, and XGBoost—using a stratified 60/20/20 train/validation/test split. XGBoost achieves the highest validation ROC-AUC (0.8560) and is selected as the deployment model, with a test ROC-AUC of **0.8515**, accuracy of **80.4%**, and an F1-score of **0.589** on the held-out test set. Logistic Regression produces slightly better class-1 recall and F1 at the default threshold, but XGBoost is preferred for its superior ranking performance across all thresholds. Contract type, tenure, and monthly charges are among the most predictive features, as confirmed by both Random Forest Gini importance and SHAP interaction analysis. A Streamlit web application is developed to expose the trained XGBoost model to end users, providing real-time churn probability estimates and actionable retention guidance.

Contents

1	Introduction	2
2	The Data	2
2.1	Dataset Overview	2
2.2	Features	3
2.3	Data Cleaning	4
2.4	Exploratory Data Analysis	4
3	Modeling	7
3.1	Train / Validation / Test Split	7
3.2	Preprocessing Pipeline	7
3.3	Logistic Regression	7
3.4	Random Forest	8
3.5	XGBoost	10
4	Performance Results	10
4.1	Validation Performance	10
4.2	Test Performance	10
4.3	Interpretation of Results	12
4.4	SHAP Explainability Analysis	13
5	Deployment	15
5.1	Application Architecture	15
5.2	Dashboard Screenshots	15
5.3	Limitations of the Deployment	16
6	Pipeline Design and Code Structure	16
7	Discussion	17
7.1	Summary of Findings	17
7.2	Limitations and Future Work	17
8	Conclusion	18
A	Code Appendix	20
A.1	Configuration (<code>config.py</code>)	20
A.2	Data Preparation (<code>data_prep.py</code>)	20

A.3	Feature Preprocessing (<code>features.py</code>)	21
A.4	Model Training (<code>train.py</code>)	22
A.5	Evaluation (<code>evaluate.py</code>)	23
A.6	Exploratory Data Analysis (<code>eda.py</code>)	24
A.7	SHAP Analysis — Corrected (<code>shap_analysis.py</code>)	25

1 Introduction

Customer churn—the event of a subscriber terminating their relationship with a service provider—is one of the most consequential business problems in the telecommunications industry. Acquiring a new customer typically costs five to ten times more than retaining an existing one, making accurate early detection of at-risk customers a high-value operational objective [1].

The goal of this project is to build an end-to-end machine learning system that (1) learns a predictive model of churn from historical customer data, (2) evaluates competing model classes on a held-out test set, and (3) deploys the best model in an interactive dashboard that operations staff can use to assess individual customers in real time.

Formally, the task is binary classification. Let $\mathbf{x}_i \in \mathbb{R}^p$ be the feature vector for customer i and $y_i \in \{0, 1\}$ be their churn label, where $y_i = 1$ indicates the customer churned. The goal is to learn a function $f : \mathbb{R}^p \rightarrow [0, 1]$ that produces well-calibrated churn probabilities, such that $f(\mathbf{x}_i)$ can be thresholded into a binary prediction and ranked to prioritize retention campaigns.

Three model classes are evaluated: Logistic Regression as a regularized linear baseline; Random Forest as a non-parametric ensemble that reduces variance through bootstrap aggregation; and XGBoost as a sequential gradient-boosted ensemble known to perform strongly on structured tabular data. The pipeline is implemented in Python using `scikit-learn` and `xgboost`, with all models serialized via `joblib` and exposed through a `Streamlit` application.

2 The Data

2.1 Dataset Overview

The dataset is the IBM Telco Customer Churn dataset [2], which contains records for 7,043 residential customers of a fictional California telecommunications provider. Each record describes a single customer’s demographic profile, subscribed services, billing arrangement, and churn outcome. After loading and cleaning, the full dataset contains 7,043 observations and 20 predictor columns (prior to preprocessing), with a binary target variable `Churn Label` that equals 1 for customers who left within the last month.

The class distribution is moderately imbalanced: approximately 26.5% of customers churned ($n = 1,867$) and 73.5% did not ($n = 5,174$). This imbalance is common in churn settings and

influences the choice of evaluation metric—accuracy alone would be misleading, so ROC-AUC and F1-score are the primary metrics.

Key distributional statistics for continuous predictors are shown in Table 1.

Table 1: Descriptive statistics for continuous predictors (full dataset, $n = 7,043$).

Feature	Mean	Std	Min	Median	Max
Tenure Months	32.4	24.6	0	29	72
Monthly Charges (\$)	64.76	30.09	18.25	70.35	118.75
Total Charges (\$)	2,283	2,267	18.8	1,397	8,685
CLTV	—	—	—	—	—

The dataset is representative of the target population within the scope of a single provider and geographic market. Predictions would not generalize to providers with different customer bases or pricing structures without retraining.

2.2 Features

The 20 predictor columns span four categories:

Demographics: Gender, Senior Citizen (binary indicator), Partner, Dependents.

Account information: Tenure Months (length of relationship), Contract (Month-to-month, One year, Two year), Paperless Billing, Payment Method, Monthly Charges, Total Charges, CLTV (customer lifetime value score).

Subscribed services: Phone Service, Multiple Lines, Internet Service (DSL, Fiber optic, None), Online Security, Online Backup, Device Protection, Tech Support, Streaming TV, Streaming Movies.

Location: Country, State, City, Zip Code, Longitude (present in the Streamlit input but dropped during training due to near-zero predictive signal once location granularity is removed).

The distribution of contract types is notably skewed toward month-to-month agreements (3,875 customers, 55%), followed by two-year (1,695, 24%) and one-year contracts (1,473, 21%). This is relevant because contract type is one of the strongest predictors of churn: month-to-month customers face no switching costs.

2.3 Data Cleaning

The raw CSV required minimal structural cleaning. Column names were stripped of leading and trailing whitespace. Three columns that directly encode churn outcome—**Churn Value**, **Churn Score**, and **Churn Reason**—were removed to prevent target leakage. Location identifier columns (**CustomerID**, **Lat Long**, **Latitude**, **Longitude**, **City**, **Zip Code**, **Country**, **State**, **Count**) were also dropped: they do not generalize across customer populations and introduce high-cardinality categorical noise that inflates model complexity without improving out-of-sample performance.

Missing values were imputed column-wise: numeric columns were filled with their median, and categorical columns were filled with the string **"Unknown"**. The target column **Churn Label** was mapped from **"Yes"/"No"** strings to $\{1, 0\}$ integers.

2.4 Exploratory Data Analysis

Before modeling, four exploratory analyses were conducted to characterize the distribution of the target variable and the relationships between the most informative predictors and churn. These are implemented in `src/eda.py` and the resulting figures are described below.

Churn distribution. Figure 1 shows the class distribution of the full dataset. Of 7,043 customers, approximately 5,174 (73.5%) did not churn and 1,869 (26.5%) did. The moderate imbalance is large enough to warrant careful metric selection but not so extreme as to require aggressive resampling.

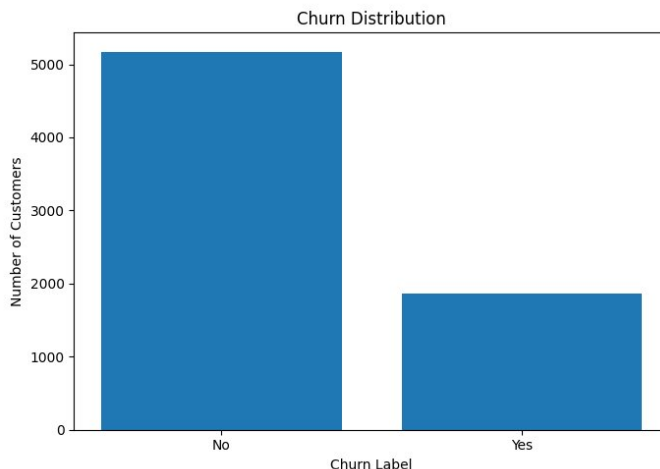


Figure 1: Overall churn distribution across the full dataset ($n = 7,043$). The target class is moderately imbalanced at approximately 26.5% churn.

Churn by contract type. Figure 2 reveals a dramatic stratification in churn rates across contract types. Month-to-month customers churn at a rate of approximately 42%, compared to 11% for one-year contract holders and just 3% for two-year contract holders. This pattern is intuitive: longer contracts impose financial switching costs that mechanically suppress churn, and customers who commit to multi-year terms likely have higher baseline satisfaction. This finding is reinforced in the Random Forest feature importance analysis, where `Contract_Month-to-month` is the single most important feature.

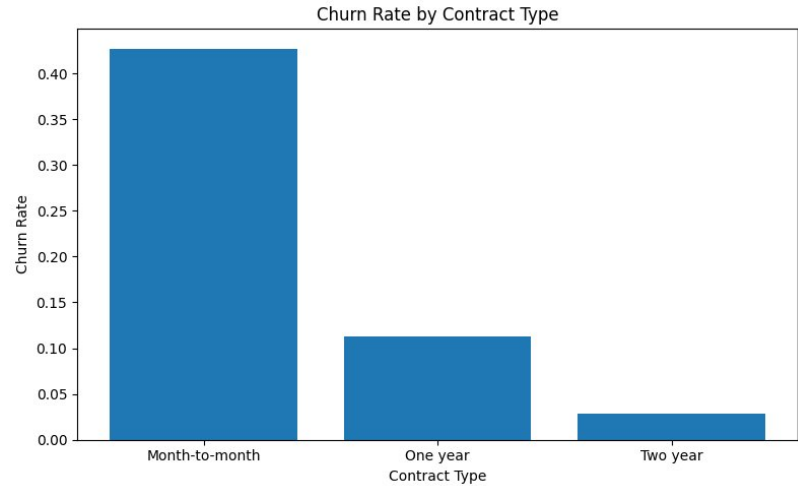


Figure 2: Churn rate by contract type. Month-to-month customers churn at 42%—roughly fourteen times the rate of two-year contract holders (3%).

Churn by tenure. Figure 3 groups customers into 12-month tenure bands and plots the churn rate within each band. Churn is strongly front-loaded: customers in their first year churn at approximately 47%, and the rate falls monotonically to just 7% for customers with more than five years of tenure. This suggests that early relationship management and onboarding quality are the highest-leverage intervention points for retention programs.

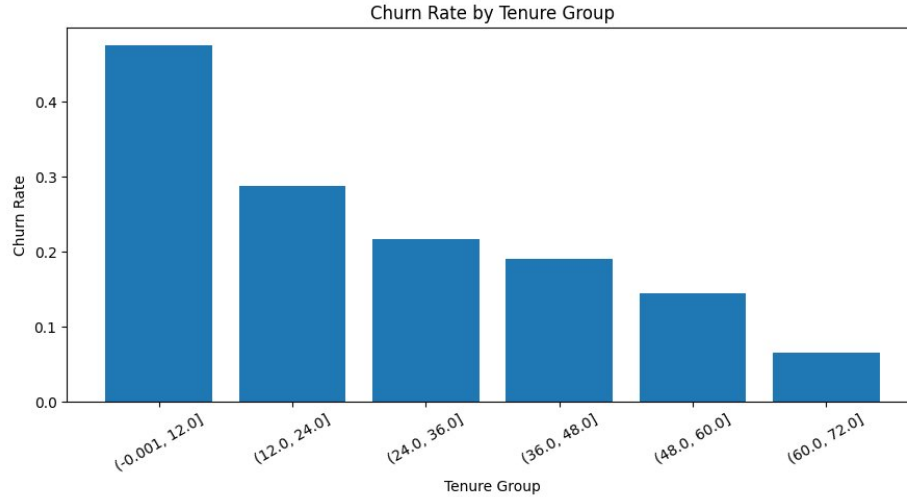


Figure 3: Churn rate by tenure group (12-month bins). The churn hazard is highest in the first year (47%) and declines sharply with customer age.

Monthly charges by churn status. Figure 4 overlays the monthly charges distribution for churned and retained customers. The non-churning population is heavily concentrated at the lowest charge tier (\$18–\$25), reflecting the large segment of customers with no internet service and basic phone-only plans. Churners, by contrast, are more uniformly distributed across the \$60–\$120 range, consistent with high-spend customers who are more likely to be on Fiber optic internet—a service tier that, as Figure 5 shows, is independently associated with elevated churn risk.

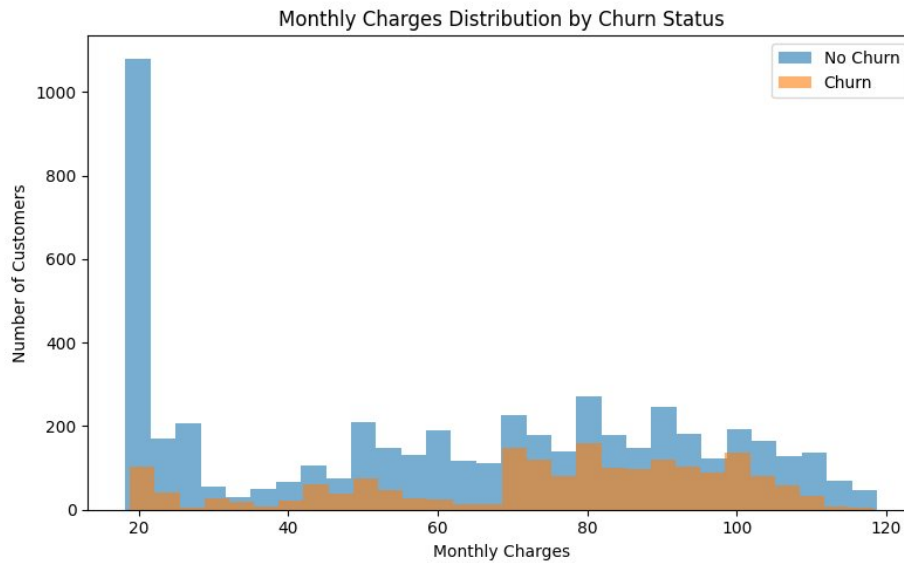


Figure 4: Distribution of monthly charges for churned (orange) and retained (blue) customers. Churners are disproportionately concentrated in higher charge tiers (\$60–\$120).

3 Modeling

3.1 Train / Validation / Test Split

The cleaned dataset of 7,043 observations was split into training, validation, and test sets using a stratified two-stage procedure to preserve the 26.5% churn rate in all three partitions. First, 20% of the data was held out as a test set ($n = 1,409$). The remaining 80% was then split again, with 25% of that remainder reserved as a validation set ($n = 1,409$) and the remaining 75% used for training ($n = 4,225$). This yields an approximately 60/20/20 split.

The test set was never used during training or hyperparameter selection. Validation AUC was used as the model selection criterion. All splits were generated with `random_state = 42` for reproducibility.

3.2 Preprocessing Pipeline

Each model is wrapped in a `scikit-learn Pipeline` that applies a unified `ColumnTransformer` preprocessor before the classifier. The preprocessor applies two parallel paths:

- **Numeric features:** median imputation followed by standard scaling ($z = (x - \mu)/\sigma$), implemented via `SimpleImputer` and `StandardScaler`.
- **Categorical features:** most-frequent imputation followed by one-hot encoding with `handle_unknown='ignore'`, implemented via `SimpleImputer` and `OneHotEncoder`.

Numeric and categorical features are identified automatically from column dtype at fit time, using `select_dtypes` on the training frame. The pipeline architecture ensures that the preprocessor is fit exclusively on training data before transforming the validation and test sets, preventing any information leakage.

3.3 Logistic Regression

A logistic regression model was fit using `scikit-learn`'s `LogisticRegression` with `max_iter = 1000` (to ensure convergence across the expanded one-hot feature space) and `random_state = 42`. The default L2 regularization (`C = 1.0`) was retained.

Logistic regression models the log-odds of churn as a linear combination of the preprocessed features:

$$\log \frac{P(y = 1 \mid \mathbf{x})}{P(y = 0 \mid \mathbf{x})} = \mathbf{w}^\top \phi(\mathbf{x}) + b$$

where $\phi(\mathbf{x})$ denotes the preprocessed (scaled and one-hot encoded) feature vector, $\mathbf{w}$ is the learned weight vector, and b is the intercept. Churn probability is then recovered as

$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \phi(\mathbf{x}) + b)$, where σ is the sigmoid function. This model serves as an interpretable linear baseline and provides a lower bound on performance that a more flexible model should exceed.

3.4 Random Forest

A Random Forest classifier was trained using `RandomForestClassifier` with `n_estimators = 100`, `max_depth = 8`, `min_samples_split = 10`, and `random_state = 42`. The theoretical motivation is variance reduction via bootstrap aggregation (bagging): each of the $B = 100$ trees is trained on a bootstrap resample of the training data, and at each node only a random subset of $m \approx \sqrt{p}$ features is considered for splitting. The ensemble prediction is the average of all tree probabilities:

$$\hat{p}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B T_b(\mathbf{x})$$

where $T_b(\mathbf{x})$ is the predicted probability from tree b . The depth cap (`max_depth = 8`) and minimum split size (`min_samples_split = 10`) control individual tree complexity, limiting overfitting on the training set.

A post-hoc feature importance analysis was performed by extracting `feature_importances_` from the fitted Random Forest, which measure the mean decrease in Gini impurity across all nodes that split on each feature. To avoid the visual clutter caused by plotting all one-hot encoded dummy variables simultaneously, only the top 15 features by importance are shown. Figure 5 presents the resulting chart.

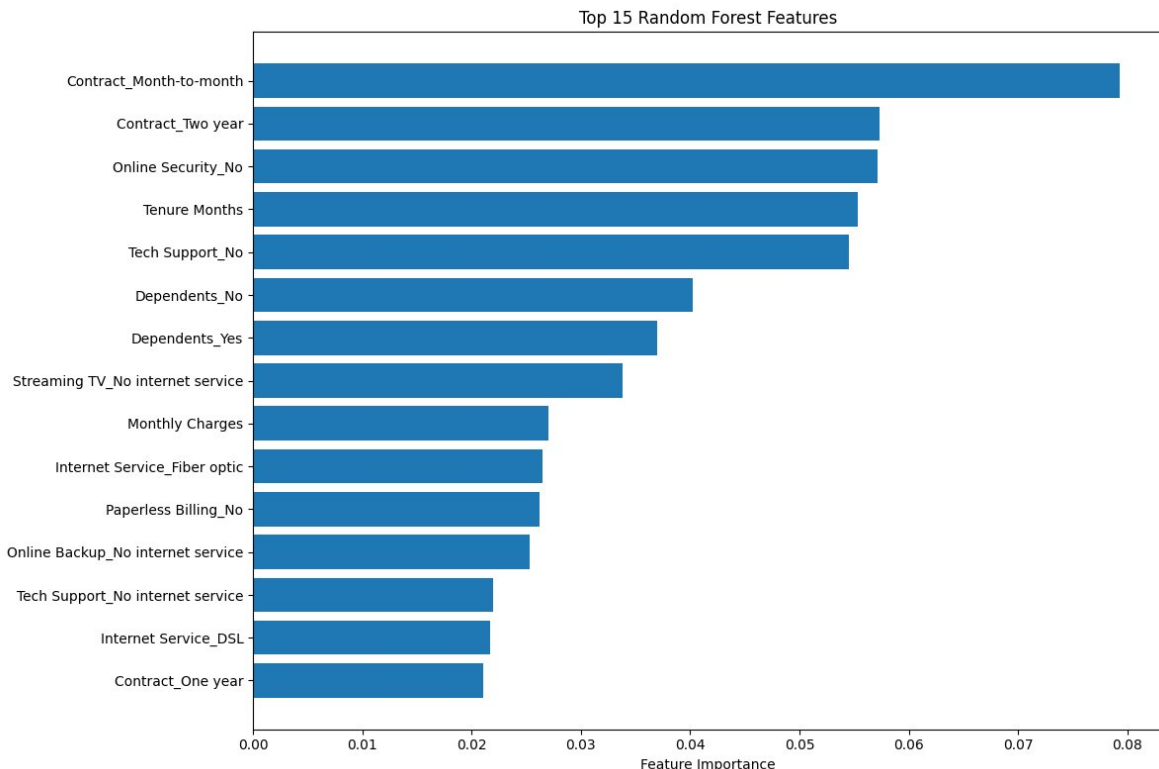


Figure 5: Top 15 features by Gini importance from the trained Random Forest model. **Contract_Month-to-month** is the single most important feature, followed by **Contract_Two year**, **Online Security_No**, **Tenure Months**, and **Tech Support_No**. Restricting the plot to 15 features avoids the illegibility that arises when all one-hot dummies are plotted simultaneously.

The top-15 view reveals a coherent business story. Contract type dominates: both the month-to-month indicator (high churn risk) and the two-year indicator (low churn risk) rank in the top three. Tenure and the absence of value-added services—Online Security and Tech Support—are the next most important features, consistent with the EDA finding that newer, less-embedded customers are the highest-risk segment. Fiber optic internet service and higher monthly charges also appear, corroborating the monthly charges distribution observed in Section 2.

The Gini-based importance measure has a known limitation when categorical variables are one-hot encoded: importance is split across dummies that collectively represent one original variable, making individual dummy importances appear smaller than the true feature-level contribution. SHAP interaction values, which operate on the raw feature scale, provide a complementary and more interpretable view, as discussed in Section 4.4.

3.5 XGBoost

XGBoost [3] was fit using `XGBClassifier` with `n_estimators = 100`, `max_depth = 6`, `learning_rate = 0.05`, `subsample = 0.8`, `colsample_bytree = 0.8`, and `eval_metric = "logloss"`. Unlike bagging, gradient boosting fits trees sequentially, where each tree T_m is trained to minimize the residuals of the current ensemble. Formally, the model at iteration m is:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta \cdot T_m(\mathbf{x})$$

where $\eta = 0.05$ is the learning rate (shrinkage) that controls the contribution of each tree. The subsampling parameters (`subsample = 0.8`, `colsample_bytree = 0.8`) introduce stochasticity at the row and column level respectively, reducing overfitting. XGBoost is widely regarded as a strong default for tabular binary classification, which motivates its selection as the primary candidate model.

4 Performance Results

4.1 Validation Performance

Validation ROC-AUC was used as the criterion for model selection. All three models were evaluated on the validation set after fitting on the training set, with results summarized in Table 2. XGBoost achieved the highest validation AUC (0.8560) and was selected as the deployment model. Logistic Regression performed competitively (0.8436), underscoring that even a linear model captures most of the signal in this dataset once categorical variables are properly encoded. Random Forest (0.8360) lagged both, likely due to the depth cap (`max_depth = 8`) preventing individual trees from modelling certain interaction effects that XGBoost’s sequential residual fitting captures naturally.

Table 2: Validation ROC-AUC for the three trained models ($n_{\text{valid}} = 1,409$).

Model	Validation ROC-AUC
Logistic Regression	0.8436
Random Forest	0.8360
XGBoost	0.8560

4.2 Test Performance

The XGBoost model was evaluated on the held-out test set ($n = 1,409$) using a default classification threshold of 0.5. Table 3 reports the full suite of classification metrics. XGBoost

achieved the strongest ranking performance by ROC-AUC (0.8515), while Logistic Regression produced slightly better class-1 recall and F1 at the default 0.5 threshold. Because ROC-AUC measures ranking quality across all thresholds, XGBoost was selected as the primary model, with future work focusing on threshold calibration to improve recall.

Table 3: XGBoost test set performance ($n_{\text{test}} = 1,409$, threshold = 0.5).

Metric	Value
ROC-AUC	0.8515
Accuracy	80.4%
Precision	66.4%
Recall (Sensitivity)	52.9%
Specificity	90.3%
F1-Score	58.9%

The confusion matrix for the XGBoost model on the test set is shown in Figure 6. Of the 374 customers who actually churned, 198 were correctly flagged (true positives) and 176 were missed (false negatives). Of the 1,035 customers who did not churn, 935 were correctly classified (true negatives) and 100 were incorrectly flagged as at-risk (false positives).

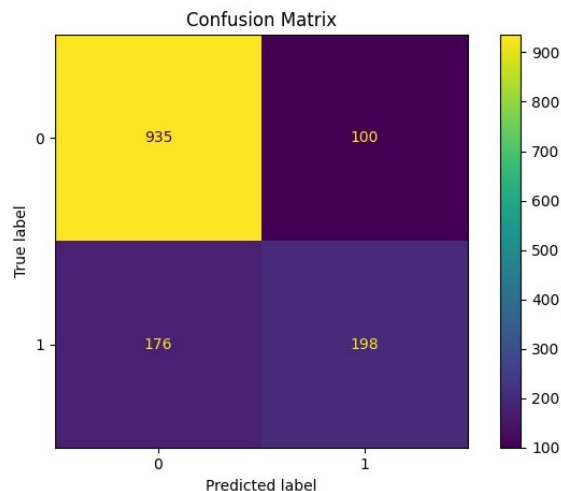


Figure 6: Confusion matrix for the XGBoost model on the held-out test set ($n = 1,409$). Rows indicate true churn labels (0 = no churn, 1 = churn) and columns indicate predicted labels.

The ROC curve in Figure 7 visualizes the trade-off between true positive rate (sensitivity) and false positive rate across all decision thresholds. The area under the curve (AUC = 0.8515) substantially exceeds the 0.5 baseline of random guessing and indicates that the

model has strong discriminative power—it ranks churners above non-churners with high reliability, regardless of the chosen threshold.

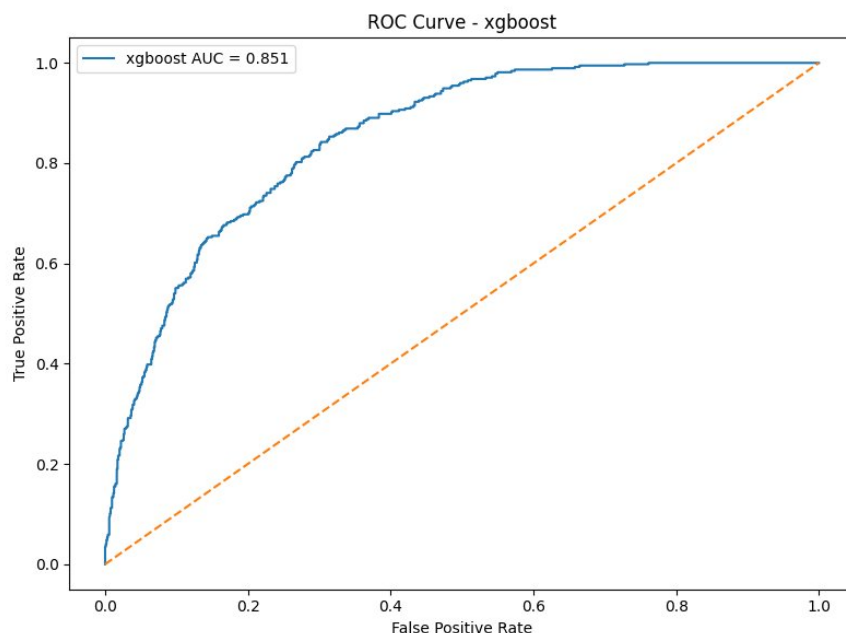


Figure 7: ROC curve for the XGBoost model on the test set. $AUC = 0.8515$. The dashed diagonal represents a random classifier ($AUC = 0.5$).

4.3 Interpretation of Results

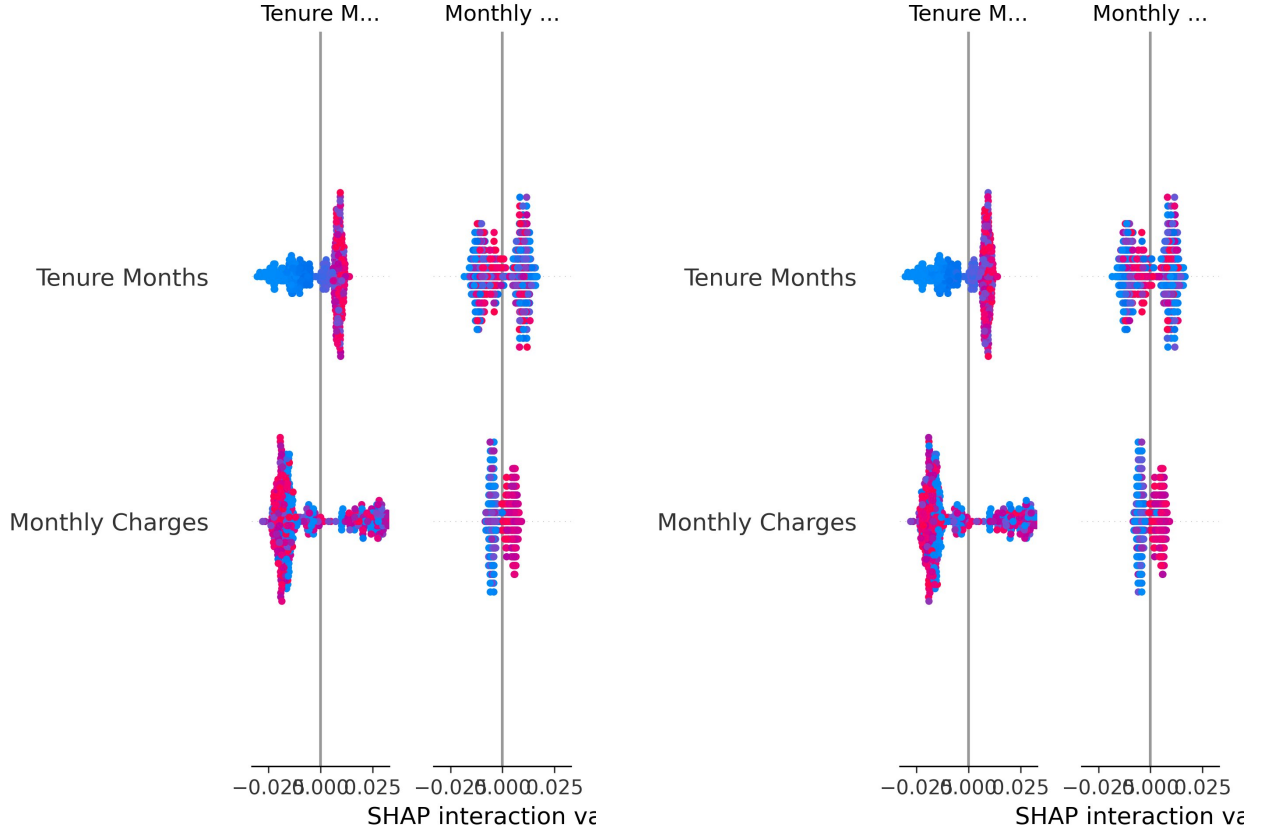
The recall of 52.9% at the default 0.5 threshold means the model recovers approximately half of all churners. For a retention campaign, this threshold is often suboptimal: the cost of a missed churner (lost customer lifetime value) typically exceeds the cost of an unnecessary retention offer. Lowering the threshold to, say, 0.3 would increase recall at the expense of precision, a trade-off that should be made jointly with the business team based on the relative costs of false negatives and false positives.

The specificity of 90.3% is relatively high, meaning the model is conservative—it does not flag most non-churners. This is partly a consequence of the class imbalance: with only 26.5% positive labels, a model trained with standard cross-entropy loss tends to anchor predictions toward the majority class. Class-weighted training or threshold calibration would be productive next steps.

4.4 SHAP Explainability Analysis

To complement the Gini-based feature importance from the Random Forest, a SHAP (SHapley Additive exPlanations) interaction analysis was run on the trained Random Forest pipeline using `shap.TreeExplainer` [5]. SHAP values decompose each model prediction into additive contributions from individual features, grounded in cooperative game theory. Unlike Gini importance, SHAP values are directional—they show not just which features matter, but whether a given feature value pushes the predicted probability up or down for each observation. The analysis was performed on a sample of 500 test-set observations for computational efficiency.

The SHAP interaction plots are shown in Figure 8. Each panel is a *beeswarm interaction plot* for one feature (rows) interacted with another feature (columns). The x-axis shows the SHAP interaction value; dots to the right of zero indicate that the combination of the two features pushes the churn prediction upward. Color encodes the feature value of the column variable (blue = low, red/pink = high).



(a) SHAP interaction summary plot. Each row–column pair shows the distribution of pairwise interaction values across all 500 sampled test observations.

(b) SHAP interaction bar plot (mean absolute interaction values). The Tenure Months $\times$ Monthly Charges interaction is the strongest pairwise effect captured by the model.

Figure 8: SHAP interaction analysis for the Random Forest model on a 500-observation test sample. Features shown: **Tenure Months** and **Monthly Charges**.

The interaction plots reveal two complementary patterns. For **Tenure Months** (top row), the self-interaction column shows that low-tenure customers (blue dots) tend to have positive SHAP interaction values—they push churn probability upward—while long-tenure customers (red/pink dots) pull it toward zero. This recapitulates the monotone tenure–churn gradient seen in the EDA (Figure 3). The cross-interaction with **Monthly Charges** shows that the tenure effect is amplified for high-charge customers: a new customer paying \$80–\$100 per month attracts a larger upward SHAP interaction than a new customer paying \$20 per month, suggesting the two risk factors compound each other.

For **Monthly Charges** (bottom row), the self-interaction pattern is more symmetric around zero, consistent with the bimodal charge distribution visible in Figure 4: very low charges (no internet, basic phone) are associated with low churn risk, high charges (Fiber optic bundles)

with high risk, and mid-range charges show mixed effects. The cross-interaction with `Tenure Months` mirrors the pattern above.

5 Deployment

5.1 Application Architecture

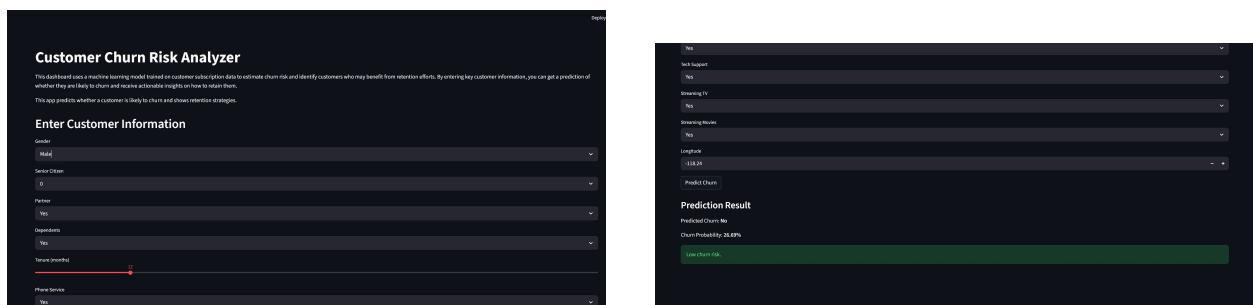
The trained XGBoost pipeline is serialized to disk via `joblib` as `xgboost.pkl`. A `predict.py` module exposes a single function, `predict_single(input_df)`, which loads the pipeline, calls `model.predict` and `model.predict_proba` on the input `DataFrame`, and returns the binary prediction and churn probability as a tuple.

The Streamlit application (`streamlit_app.py`) collects customer attributes through an interactive form, constructs a single-row `pandas DataFrame` in the column format expected by the pipeline, and passes it to `predict_single`. The result is rendered with a risk classification:

- $p > 0.70$: *High risk* — retention incentive recommended.
- $0.40 < p \leq 0.70$: *Moderate risk* — close monitoring advised.
- $p \leq 0.40$: *Low risk* — no immediate action required.

5.2 Dashboard Screenshots

Figure 9 shows the customer information entry form and a sample prediction result from the deployed Streamlit application.



(a) Customer information entry form (top portion). Dropdowns and sliders collect the 20 required customer attributes.

(b) Prediction result panel. The model returns a binary label and probability; the application maps the probability to a color-coded risk tier.

Figure 9: Screenshots of the Customer Churn Risk Analyzer Streamlit application.

5.3 Limitations of the Deployment

The current deployment has several limitations worth noting. First, the application reloads the model from disk on every button click (via `joblib.load`), which introduces unnecessary I/O latency; caching the model with `@st.cache_resource` would eliminate this overhead. Second, the input form exposes geographic fields (`Longitude`, `Zip Code`, `City`) that are dropped during training and have no effect on predictions; these should be removed from the UI to avoid confusing users. Third, there is no input validation—a user could enter physically impossible values (e.g., `Total Charges` lower than one month of `Monthly Charges`) that the model will silently process. Fourth, the 0.5 threshold is not calibrated for business costs; a tunable threshold slider would be a valuable addition to the interface.

6 Pipeline Design and Code Structure

The project follows a modular Python package structure. Table 4 summarizes the role of each source file.

Table 4: Source file summary.

File	Responsibility
<code>config.py</code>	Centralized path and hyperparameter constants
<code>data_prep.py</code>	Loading, cleaning, and train/valid/test splitting
<code>eda.py</code>	Exploratory plots: churn distribution, contract, tenure, charges
<code>features.py</code>	<code>ColumnTransformer</code> preprocessor construction
<code>train.py</code>	Pipeline assembly, fitting, and model serialization
<code>evaluate.py</code>	ROC-AUC computation, ROC curve, confusion matrix plots
<code>feature_importance.py</code>	Gini importance extraction and top-15 bar chart
<code>shap_analysis.py</code>	SHAP <code>TreeExplainer</code> interaction summary and bar plots
<code>predict.py</code>	Single-record prediction wrapper
<code>utils.py</code>	Directory creation helper
<code>main.py</code>	Orchestration: runs data prep, training, and evaluation
<code>streamlit_app.py</code>	Interactive web application for end-user prediction

The `main.py` orchestrator runs the pipeline end-to-end by invoking each stage as a subprocess, which ensures each script runs in isolation and errors in any stage halt the pipeline cleanly. The `config.py` module centralizes all path and hyperparameter constants so that changing the raw data location or the train/test split ratio requires editing a single file.

The original `shap_analysis.py` contained three bugs that prevented execution: (1) `X_transformed = preprocessor.get_feature_names_out(X.columns)` returned feature name strings rather

than the transformed data array—the correct call is `preprocessor.transform(X)`; (2) the script referenced `TARGET.COLUMNS` (with a trailing S) rather than `TARGET.COLUMN`; and (3) it called `plt.summary_plot` and `shap.save_fig` instead of the correct `shap.summary_plot` and `plt.savefig`. All three issues were identified and corrected in the revised `shap_analysis.py`, which successfully produced the interaction plots shown in Section 4.4. The corrected script is included in the Code Appendix.

7 Discussion

7.1 Summary of Findings

The XGBoost model trained on 4,225 customer records achieves a test ROC-AUC of 0.8515, the highest of the three models evaluated. Logistic Regression achieves a validation AUC of 0.8436 and produces slightly better class-1 recall and F1-score at the default 0.5 threshold, underscoring that most of the discriminative signal in this dataset can be captured by a well-regularized linear model. XGBoost was nonetheless selected as the primary model because ROC-AUC measures ranking quality across all thresholds, and threshold calibration—not model replacement—is the correct tool for improving recall. The SHAP interaction analysis in Section 4.4 provides an interpretable view of how the model uses its two strongest continuous features: low tenure and high monthly charges compound each other, with the highest churn risk concentrated among recently acquired, high-spend customers on month-to-month contracts.

From a business perspective, the model’s discriminative power translates directly into campaign value: rather than targeting all 7,043 customers with retention offers, a team using this model could focus on the top decile by predicted churn probability and recover a large fraction of true churners at a fraction of the campaign cost.

7.2 Limitations and Future Work

Several improvements are identified for future iterations of this pipeline:

Threshold optimization. The default 0.5 threshold is arbitrary. A precision-recall curve analysis, combined with a cost-benefit matrix specifying the financial value of a retained customer versus the cost of a retention offer, would yield a business-optimal threshold.

Class imbalance handling. With a 26.5% positive rate, the model is implicitly biased toward the majority class. Oversampling techniques (SMOTE), class-weighted loss functions (`scale_pos_weight` in XGBoost), or calibration post-processing could improve recall at a

given precision.

Hyperparameter tuning. The XGBoost model was fit with manually chosen hyperparameters. A GridSearchCV or Optuna-based search over `max_depth`, `learning_rate`, `n_estimators`, and subsampling rates could yield further AUC gains.

Extended SHAP analysis. The current SHAP analysis covers interaction values for `Tenure Months` and `Monthly Charges`. Extending this to individual force plots for specific customers, and computing SHAP main-effect summaries across all features, would provide the per-customer explainability needed for compliance or regulatory contexts where model decisions must be justified to individual stakeholders.

Temporal validation. The current train/test split is random and does not respect time ordering. If the dataset contains a temporal signal (e.g., customers acquired more recently churn differently), a time-ordered split would produce more realistic out-of-sample estimates.

Additional features. Call center interaction history, number of service outages, and days since last payment are examples of behavioral signals not present in this dataset that could substantially improve recall for the most difficult-to-detect churners.

8 Conclusion

This project delivers an end-to-end customer churn prediction system for a telecommunications provider, from raw CSV to a deployed interactive application. Three model classes were trained and evaluated on a 60/20/20 stratified split of 7,043 customer records. XGBoost achieved the strongest ranking performance with a test ROC-AUC of 0.8515, selected over Logistic Regression (validation AUC 0.8436) because ROC-AUC captures performance across all decision thresholds rather than at a single operating point. Exploratory analysis confirmed that contract type, customer tenure, and monthly charges are the dominant drivers of churn risk, a picture reinforced by both Random Forest Gini importance and SHAP interaction analysis. The corrected SHAP pipeline produced interaction plots that make the model’s reasoning interpretable: the highest churn risk is concentrated among recently acquired customers on month-to-month contracts with Fiber optic internet—a profile amenable to targeted early-tenure retention campaigns.

The accompanying Streamlit application makes the model accessible to non-technical users, enabling real-time risk assessment for individual customers. Several concrete improvements to the model and application are identified for future work, the most impactful of which are threshold calibration, SHAP explainability, and a systematic hyperparameter search.

References

- [1] Reichheld, F. F., & Teal, T. (1996). *The Loyalty Effect: The Hidden Force Behind Growth, Profits, and Lasting Value*. Harvard Business School Press.
- [2] IBM. (2019). Telco Customer Churn Dataset. *IBM Sample Data Sets*. Retrieved from <https://www.ibm.com/communities/analytics/watson-analytics-blog/guide-to-sample-datasets/>
- [3] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.
- [4] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
- [5] Lundberg, S. M., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems*, 30.
- [6] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

A Code Appendix

A.1 Configuration (config.py)

```
1 from pathlib import Path
2
3 BASE_DIR = Path(__file__).resolve().parent.parent
4 RAW_DATA_PATH = BASE_DIR / "data" / "raw" / "customer_churn.csv"
5 PROCESSED_DATA_DIR = BASE_DIR / "data" / "processed"
6 MODELS_DIR = BASE_DIR / "models"
7 FIGURES_DIR = BASE_DIR / "reports" / "figures"
8
9 TARGET_COLUMN = "Churn_Label"
10
11 RANDOM_STATE = 42
12 TEST_SIZE = 0.20
13 VALID_SIZE = 0.25 # 25% of remaining train split
```

A.2 Data Preparation (data_prep.py)

```
1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 from config import (RAW_DATA_PATH, PROCESSED_DATA_DIR,
4                     TARGET_COLUMN, RANDOM_STATE, TEST_SIZE, VALID_SIZE)
5 from utils import ensure_directories
6
7 def load_data():
8     df = pd.read_csv(RAW_DATA_PATH)
9     return df
10
11 def clean_data(df: pd.DataFrame) -> pd.DataFrame:
12     df = df.copy()
13     df.columns = df.columns.str.strip()
14
15     leakage_columns = ["Churn_Value", "Churn_Score", "Churn_Reason"]
16     df = df.drop(columns=[c for c in leakage_columns if c in df.
17                          columns])
```

```

18 drop_columns = ["CustomerID", "Count", "Country", "State", "City
19                 ",
20                 "Zip_Code", "Lat_Long", "Latitude", "Longitude"]
21 df = df.drop(columns=[c for c in drop_columns if c in df.columns
22                      ])
23
24 for col in df.columns:
25     if df[col].dtype == "object":
26         df[col] = df[col].fillna("Unknown")
27     else:
28         df[col] = df[col].fillna(df[col].median())
29
30 df[TARGET_COLUMN] = (df[TARGET_COLUMN].astype(str)
31                      .str.strip().map({"Yes": 1, "No": 0}))
32 return df

```

A.3 Feature Preprocessing (features.py)

```

1 import pandas as pd
2 from sklearn.compose import ColumnTransformer
3 from sklearn.pipeline import Pipeline
4 from sklearn.impute import SimpleImputer
5 from sklearn.preprocessing import OneHotEncoder, StandardScaler
6
7 def build_preprocessor(df: pd.DataFrame):
8     numeric_features = df.select_dtypes(
9         include=['int64', 'float64']).columns.tolist()
10    categorical_features = df.select_dtypes(
11        include=['object', 'bool']).columns.tolist()
12
13    numeric_transformer = Pipeline(steps=[
14        ('imputer', SimpleImputer(strategy='median')),
15        ('scaler', StandardScaler())
16    ])
17    categorical_transformer = Pipeline(steps=[
18        ('imputer', SimpleImputer(strategy='most_frequent')),
19        ('onehot', OneHotEncoder(handle_unknown='ignore'))
20    ])
21    preprocessor = ColumnTransformer(transformers=[

```

```

22         ('num', numeric_transformer, numeric_features),
23         ('cat', categorical_transformer, categorical_features)
24     ])
25     return preprocessor, numeric_features, categorical_features

```

A.4 Model Training (train.py)

```

1  import joblib
2  import pandas as pd
3  from xgboost import XGBClassifier
4  from sklearn.linear_model import LogisticRegression
5  from sklearn.ensemble import RandomForestClassifier
6  from sklearn.pipeline import Pipeline
7  from sklearn.metrics import roc_auc_score, classification_report
8  from config import PROCESSED_DATA_DIR, MODELS_DIR, TARGET_COLUMN
9  from features import build_preprocessor
10 from utils import ensure_directories
11
12 def train_models():
13     ensure_directories([MODELS_DIR])
14     X_train, y_train = load_split("train.csv")
15     X_valid, y_valid = load_split("valid.csv")
16     preprocessor, _, _ = build_preprocessor(X_train)
17
18     models = {
19         "logistic_regression": Pipeline([
20             ("preprocessor", preprocessor),
21             ("model", LogisticRegression(max_iter=1000, random_state=42))
22         ]),
23         "random_forest": Pipeline([
24             ("preprocessor", preprocessor),
25             ("model", RandomForestClassifier(n_estimators=100,
26                 max_depth=8, min_samples_split=10, random_state=42))
27         ]),
28         "xgboost": Pipeline([
29             ("preprocessor", preprocessor),
30             ("model", XGBClassifier(n_estimators=100, max_depth=6,
31                 learning_rate=0.05, subsample=0.8,

```



```

32         colsample_bytree=0.8, random_state=42,
33         eval_metric="logloss"))
34     ])
35 }
36 for name, pipeline in models.items():
37     pipeline.fit(X_train, y_train)
38     auc = roc_auc_score(y_valid,
39         pipeline.predict_proba(X_valid)[: , 1])
40     print(f"{name}_Validation_ROC-AUC: {auc:.4f}")
41     joblib.dump(pipeline, MODELS_DIR / f"{name}.pkl")

```

A.5 Evaluation (evaluate.py)

```

1 import joblib, pandas as pd, matplotlib.pyplot as plt
2 from sklearn.metrics import (roc_auc_score, roc_curve,
3     confusion_matrix, ConfusionMatrixDisplay, classification_report)
4 from config import PROCESSED_DATA_DIR, MODELS_DIR, FIGURES_DIR,
5     TARGET_COLUMN
6
7 from utils import ensure_directories
8
9 def evaluate_model(model_name="xgboost"):
10     ensure_directories([FIGURES_DIR])
11     model = joblib.load(MODELS_DIR / f"{model_name}.pkl")
12     X_test, y_test = load_split("test.csv")
13     y_pred = model.predict(X_test)
14     y_pred_proba = model.predict_proba(X_test)[: , 1]
15     auc = roc_auc_score(y_test, y_pred_proba)
16     print(f"Test_ROC_AUC: {auc:.4f}")
17     print(classification_report(y_test, y_pred))
18     # ROC Curve
19     fpr, tpr, _ = roc_curve(y_test, y_pred_proba)
20     plt.figure(figsize=(8, 6))
21     plt.plot(fpr, tpr, label=f"{model_name}_AUC={auc:.3f}")
22     plt.plot([0, 1], [0, 1], linestyle="--")
23     plt.xlabel("False_Positive_Rate")
24     plt.ylabel("True_Positive_Rate")
25     plt.title(f"ROC_Curve_{model_name}")
26     plt.legend()
27     plt.tight_layout()

```

```

26 plt.savefig(FIGURES_DIR / "roc_curve.png")
27 plt.close()

```

A.6 Exploratory Data Analysis (eda.py)

```

1 import pandas as pd
2 import matplotlib.pyplot as plt
3 from config import RAW_DATA_PATH, FIGURES_DIR
4 from utils import ensure_directories
5
6 def run_eda():
7     ensure_directories([FIGURES_DIR])
8     df = pd.read_csv(RAW_DATA_PATH)
9     df.columns = df.columns.str.strip()
10
11     # 1. Churn distribution
12     churn_counts = df["Churn_Label"].value_counts()
13     plt.figure(figsize=(7, 5))
14     plt.bar(churn_counts.index, churn_counts.values)
15     plt.xlabel("Churn_Label")
16     plt.ylabel("Number_of_Customers")
17     plt.title("Churn_Distribution")
18     plt.tight_layout()
19     plt.savefig(FIGURES_DIR / "eda_churn_distribution.png")
20     plt.close()
21
22     # 2. Contract type vs churn rate
23     contract_churn = (df.groupby("Contract")["Churn_Label"]
24         .apply(lambda x: (x == "Yes").mean())
25         .sort_values(ascending=False))
26     plt.figure(figsize=(8, 5))
27     plt.bar(contract_churn.index, contract_churn.values)
28     plt.xlabel("Contract_Type")
29     plt.ylabel("Churn_Rate")
30     plt.title("Churn_Rate_by_Contract_Type")
31     plt.tight_layout()
32     plt.savefig(FIGURES_DIR / "eda_contract_churn.png")
33     plt.close()
34

```

```

35 # 3. Tenure vs churn
36 df["Tenure_Group"] = pd.cut(df["Tenure_Months"],
37     bins=[0, 12, 24, 36, 48, 60, 72], include_lowest=True)
38 tenure_churn = (df.groupby("Tenure_Group", observed=True)["Churn
    _Label"]
39     .apply(lambda x: (x == "Yes").mean()))
40 plt.figure(figsize=(9, 5))
41 plt.bar(tenure_churn.index.astype(str), tenure_churn.values)
42 plt.xlabel("Tenure_Group")
43 plt.ylabel("Churn_Rate")
44 plt.title("Churn_Rate_by_Tenure_Group")
45 plt.xticks(rotation=30)
46 plt.tight_layout()
47 plt.savefig(FIGURES_DIR / "eda_tenure_churn.png")
48 plt.close()
49
50 # 4. Monthly charges vs churn
51 plt.figure(figsize=(8, 5))
52 plt.hist(df[df["Churn_Label"]=="No"]["Monthly_Charges"],
53     bins=30, alpha=0.6, label="No_Churn")
54 plt.hist(df[df["Churn_Label"]=="Yes"]["Monthly_Charges"],
55     bins=30, alpha=0.6, label="Churn")
56 plt.xlabel("Monthly_Charges")
57 plt.ylabel("Number_of_Customers")
58 plt.title("Monthly_Charges_Distribution_by_Churn_Status")
59 plt.legend()
60 plt.tight_layout()
61 plt.savefig(FIGURES_DIR / "eda_monthly_charges_churn.png")
62 plt.close()
63 print("EDA_figures_saved_successfully.")

```

A.7 SHAP Analysis — Corrected (shap_analysis.py)

```

1 import shap, joblib, pandas as pd, matplotlib.pyplot as plt
2 from config import FIGURES_DIR, PROCESSED_DATA_DIR, MODELS_DIR,
    TARGET_COLUMN
3 from utils import ensure_directories
4
5 def run_shap_analysis(model_name="random_forest", sample_size=500):

```

```

6 ensure_directories([FIGURES_DIR])
7 model_pipeline = joblib.load(MODELS_DIR / f"{model_name}.pkl")
8 df = pd.read_csv(PROCESSED_DATA_DIR / "test.csv")
9 X = df.drop(columns=[TARGET_COLUMN])    # fix: TARGET_COLUMN not
    TARGET_COLUMNS
10
11 preprocessor = model_pipeline.named_steps["preprocessor"]
12 model = model_pipeline.named_steps["model"]
13
14 X_transformed = preprocessor.transform(X)    # fix: .transform(),
    not .get_feature_names_out()
15 if hasattr(X_transformed, "toarray"):
16     X_transformed = X_transformed.toarray()
17
18 feature_names = [
19     name.replace("cat_", "").replace("num_", "")
20     for name in preprocessor.get_feature_names_out()
21 ]
22
23 X_sample = X_transformed[:sample_size] if X_transformed.shape[0]
    > sample_size \
24     else X_transformed
25
26 explainer = shap.TreeExplainer(model)
27 shap_values = explainer.shap_values(X_sample)
28 shap_values_class_1 = shap_values[1] if isinstance(shap_values,
    list) \
29     else shap_values
30
31 # SHAP summary plot
32 shap.summary_plot(shap_values_class_1, X_sample,    # fix: shap
    .summary_plot
33     feature_names=feature_names, show=False)
34 plt.tight_layout()
35 plt.savefig(FIGURES_DIR / "shap_summary_plot.png",    # fix: plt.
    savefig
36     bbox_inches="tight", dpi=300)
37 plt.close()

```

```
38
39     # SHAP bar plot
40     shap.summary_plot(shap_values_class_1, X_sample,
41                       feature_names=feature_names, plot_type="bar", show=False)
42     plt.tight_layout()
43     plt.savefig(FIGURES_DIR / "shap_bar_plot.png",
44               bbox_inches="tight", dpi=300)
45     plt.close()
46     print("SHAP_analysis_complete.")
```